MnemosyneOS

A capability-based research OS
with deterministic service replay



Review I
Executable prototype / October 2026

Problem statement

A failed service must recover state while current permissions still apply.

A restart loses state. An incomplete replay can diverge.
An unchecked recovery proposal can exceed its authority.

Research question: can bounded checkpoint/replay restore a service predictably while its peers continue running?

Objectives

Isolation	Private user address spaces, preemptive scheduling and capability-checked IPC.
Persistence	Atomic filesystem roots, snapshots and a recovery bundle that survives reboot.
Recovery	Native COW replay and a fresh Wasm worker with matching state and effects.
Authority	A typed proposal must pass deterministic policy checks before execution.

Acceptance uses executable guest scenarios and failure-oriented shared-code tests.

Background: kernel structure and state

Study	Design lesson	Project boundary
seL4 **Klein et al., 2009**	Keep authority and kernel mechanisms explicit, with precise correctness obligations.	Tests and runtime checks do not inherit seL4’s formal proof.
Theseus **Boos et al., 2020**	State ownership and component boundaries influence recovery.	Native services are isolated, but filesystem and runtime management still share init.

 Primary papers inform the design. MnemosyneOS uses its own kernel implementation.

Background: recovery and verification

Study	Design lesson	Current application
RIFL **Lee et al., 2015**	Retries need operation identity and durable completion handling.	External exactly-once effects remain outside the prototype.
CrashMonkey / B3 **Mohan et al., 2018**	Bound failure cases and inspect recovered persistent state.	Interrupt commit operations and corrupt stored roots in tests.
LLM-Modulo **Gundawar et al., 2024**	Separate proposal generation from external validity checks.	The model proposes. Trusted code checks a narrow recovery action.

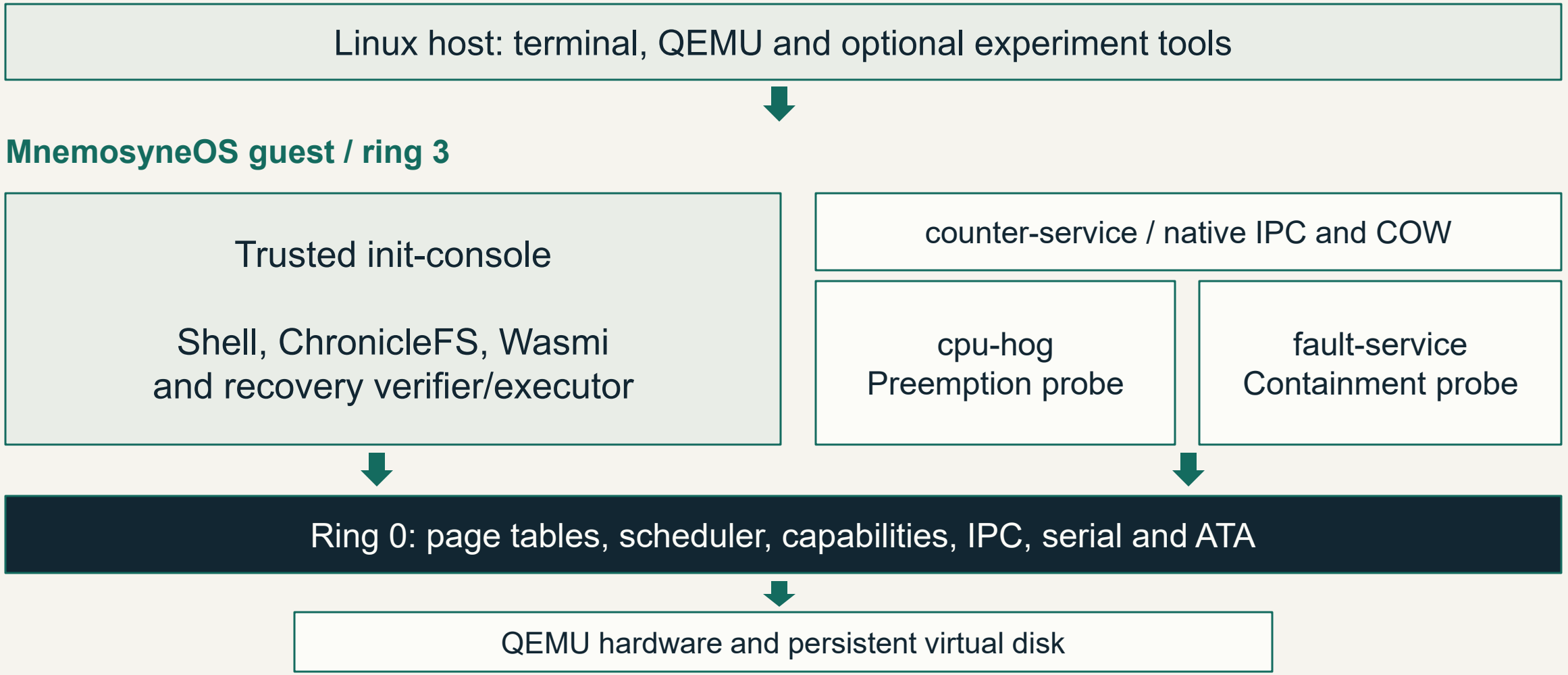
These are design implications, not claims of reproduced paper evaluations.

Reuse and project contribution

Upstream component	Reused implementation	Project integration
Limine	Boot protocol and bootloader	Kernel startup and physical memory setup
Rust OSDev crates	CPU tables, serial, PIC and heaps	Page mappings, trap ABI and task scheduling
Wasmi 2.0.0	Validation, interpretation and fuel	Restricted profile, checkpoint and replay binding
Project code	Original integration and protocols	Capabilities, ChronicleFS, recovery policy and shell

Pinned versions, license notices and adaptations are recorded in THIRD_PARTY.md.

Current architecture



 Each native task has a private address space. Several trusted services still share init.

System design contracts

Memory and CPU	One x86-64 CPU, private page tables, guarded stacks and integer contexts.
Authority	Kernel-owned per-task tables, typed rights, generation handles and checked buffers.
Storage	Dedicated 16-MiB disk, immutable roots and explicit flush ordering.
Wasm execution	One admitted integer fixture, 64-KiB memory and 10,000 fuel per request.

The narrow profile makes state, authority and failure behavior observable.

Algorithm: scheduling and IPC

Timer or blocking syscall

Save the current task context

↓

Wake sleepers; select a ready task

↓

Switch page table and restore context

No voluntary yield is required.

Capability IPC

Resolve caller authority and buffer

↓

Queue a bounded, sender-stamped message

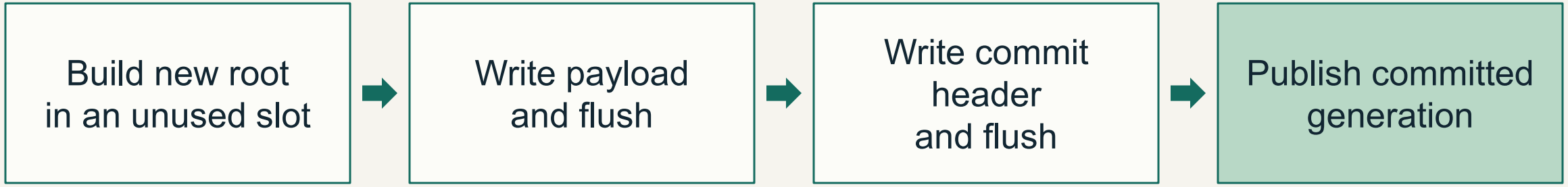
↓

Wake receiver or block until input

Invalid rights or full queues return errors.

 100-Hz round robin keeps the console running while cpu-hog never yields.

Algorithm: filesystem commit



Mount after reboot

Validate headers, payload checksums and structure.
Select the newest valid root.

64 retained generations on a 16-MiB disk.
Full volumes refuse new commits; old roots remain intact.

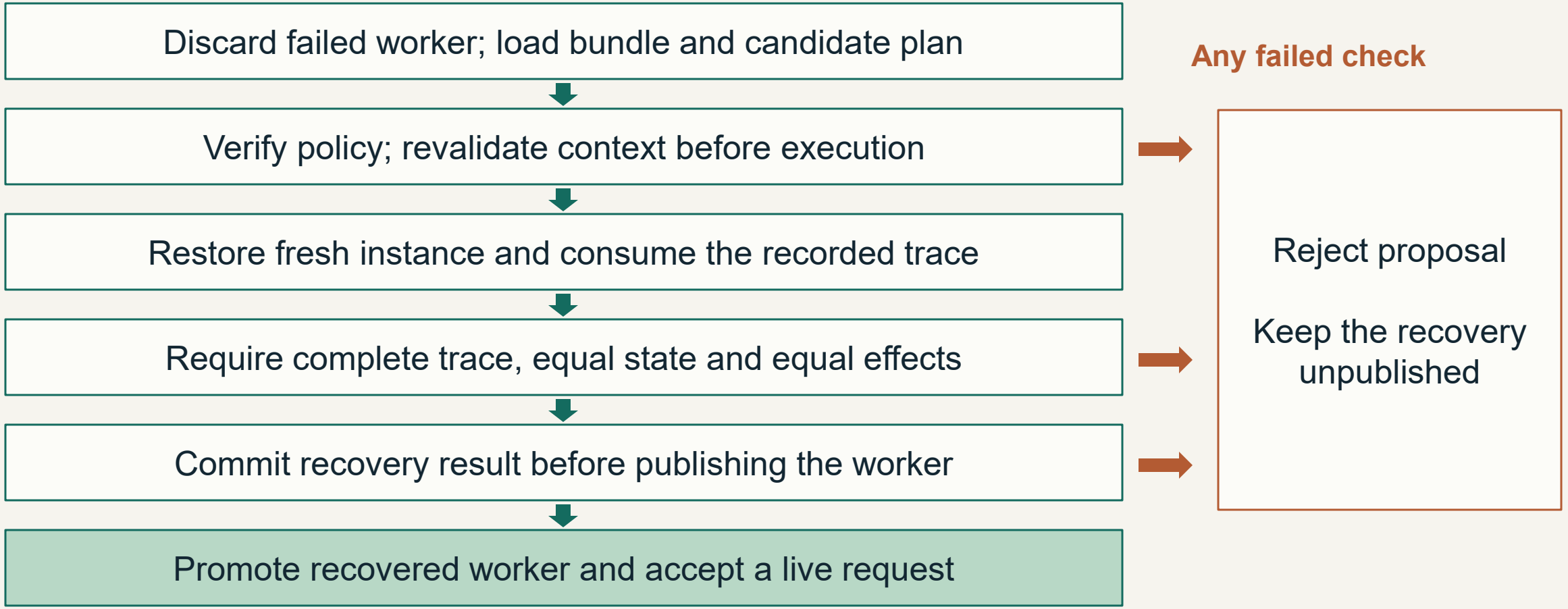
On ambiguous I/O failure, the mount refuses further operations until remount.

Two checkpoint paths

Property	Native counter service	Wasm worker
Boundary	Blocked IPC receive	Completed request handler
Saved state	Registers and page mappings	Linear memory and mutable global
Copy mechanism	Page protection and copy-on-write	Full state copy
Replay input	Two recorded IPC requests	Requests, clock and random values
Success check	Equal replies; stale handles fail	Complete trace; equal state and effects

Native COW and Wasm snapshots are separate implementations with restricted scopes.

Algorithm: policy-checked recovery



 A candidate remains unpublished until replay checks and result persistence succeed.

Typed recovery proposal

```
{
  "version": 1,
  "action": "restore-replay",
  "service": "kv-service",
  "epoch": 6,
  "checkpoint-sha256": "199be84b...",
  "max-events": 60,
  "fuel-per-request": 10000,
  "memory-pages": 1,
  "requested-rights": 3,
  "require-rollback": true
}
```

Checks before execution

Identity and epoch
Current authority
No extra privileges
Event, fuel and memory limits
Rollback availability

Display example: digest shortened. Use the full plan from plan-show for execution.

Implemented kernel functionality

Module	Working behavior	Visible evidence
Boot and memory	Rust kernel, page tables, heaps and faults	Boot checks pass
Task scheduling	Four ring-3 tasks and timer preemption	ps; hog preemptions
Capability IPC	Typed handles, bounded queues and buffers	ipc-demo; cap-demo
Fault containment	Faulting worker dies; peers continue	Three CPU fault probes
Native recovery	COW restore, endpoint refresh and replay	cow-demo passes

Compiled-in tasks on one CPU; general application admission remains future work.

Implemented storage and recovery

Module	Working behavior	Current limit
ChronicleFS	Files, atomic roots, snapshots and reboot mount	Flat names; finite generations
Guest Wasmi	Fuel-limited integer workload in userspace	One admitted fixture
Durable replay	Saved bundle; fresh replay; resumed requests	In-memory effect comparison
Recovery policy	Typed action and execution-time checks	No formal soundness proof

These modules share the trusted init address space in this prototype.

Executable prototype evidence

6 / 6

guest demo stages

12 / 60

requests / recorded events

2 boots

persistence and recovery

```
guest-demo: passed
state-equal: true     effects-equal: true
unsafe plan: PrivilegeAmplification
recovered live request: value=180
```

 Recorded native-Linux guest run, 9 October 2026. Source and ISO hash are in the notes.

Implementation progress and Review I

6 / 6

bounded prototype
acceptance gates evidenced

Full-project completion is not yet quantified.

Agree final criteria and weights.
Attach evidence to completed criteria.
Keep unfinished evaluation explicit.

Demo coverage and implementation effort measure different things.

 A scope decision is still needed before asserting the required 60–70% completion.

Proposed module division

Workstream	Owner	Modules and review responsibility
Kernel and scheduling	Unassigned	Boot, memory, syscalls and preemption demonstration
Security and native recovery	Unassigned	Capabilities, IPC, COW and stale-handle rejection
Storage and Wasm	Unassigned	ChronicleFS, durable bundles and reboot recovery
Policy and evaluation	Unassigned	Verifier, AI adapter, fault campaigns and dashboard

Suggested workstreams. Actual names, ownership and contribution records are pending.

Live demo: boot and execute the core

```
Linux host, inside the demo bundle:

python3 tools/run-image.py --self-test
python3 tools/run-image.py \
  --disk build/review-one.img

MnemosyneOS guest console:

about
ps
demo
ls
snapshots
```

What the professor sees

Own kernel and console
Protected user tasks
Preemption and IPC
Contained faults
Filesystem snapshots
Checked service recovery

Use an existing non-Azure Linux VM. QEMU runs one CPU and 256 MiB without a guest NIC.

Live demo: persistence and recovery

```
Guest, after demo:
write professor-note.txt survives-reboot
cat professor-note.txt
shutdown

Host, boot the same disk:
python3 tools/run-image.py \
  --disk build/review-one.img

Guest:
cat professor-note.txt
recover
kv-add 1 2
```

Expected outcome

The file still contains survives-reboot

Replay checks pass.

The fixed KV fixture accepts a new request and returns 180.

 The saved bundle is durable. Later live KV changes are volatile in this demonstration.

Remaining implementation and evaluation

Service boundaries	Separate storage/runtime management from init and broaden task lifecycle coverage.
Recovery coverage	Admit additional workloads and exercise more state, trace and storage failures.
Measured evaluation	Collect repeated IPC, capability, Wasm, checkpoint and recovery measurements.
Control plane	Connect current guest evidence to the dashboard and compare model proposals with a deterministic baseline.

 The current 10-ms QEMU timer cannot support fine-grained performance conclusions.

Research contribution to evaluate

Predictable recovery under current authority and bounded execution.

The integrated prototype is executable.
The next evidence must quantify correctness coverage and recovery cost.

Review decisions: confirm the final scope, assign team ownership and agree the completion criteria.

Review questions

Why a custom kernel?	Kernel construction is part of the project, and it exposes the authority and recovery mechanisms.
Why a browser UI?	It is optional host experiment tooling. The guest OS has its own serial console.
Does AI run the OS?	No. A model may propose a typed plan; trusted code checks and executes an allowed action.
Is it formally verified?	No. Runtime policy checks and failure tests support the restricted prototype claims.

 Current limits include no SMP, guest networking, desktop, arbitrary loader or external exactly-once delivery.

References and reproducibility

seL4	Klein et al., SOSP 2009
Theseus	Boos et al., OSDI 2020
RIFL	Lee et al., SOSP 2015
CrashMonkey / B3	Mohan et al., OSDI 2018
LLM-Modulo	Gundawar et al., arXiv 2024
Project evidence	Source, recorded logs, build pins and cloud runbook

 Full paper links, source paths and evidence identities appear in slide notes and the review guide.